

RAG with Python — 1 Month Intern Learning Plan

Daily learning plan, practical assignments, weekly reviews and demo project

Primary goal

By the end of the month, build and explain an end-to-end RAG application: document upload → text extraction → chunking → embeddings → vector database → retrieval → LLM answer with sources.

Learning rule

Do not move to the next topic until you have built something small with the current topic.

Week 1 — Python, Database & API Foundation

Day	Learn	Hands-on	Deliverable
Day 1	Python basics: variables, lists, dictionaries, loops	Build a small CLI program	Python basics demo
Day 2	Functions, classes, modules	Create a reusable Python utility	Utility project
Day 3	async/await, virtual environments, packages	Build an async request example	Async demo
Day 4	File handling, PDF/text, JSON	Read a document and extract structured data	Document reader
Day 5	try/except, logging, error handling	Add robust errors to previous work	Error-handling demo
Day 6	SQL or MongoDB basics, CRUD	Create document/user CRUD	CRUD application
Day 7	GET/POST, headers, API keys, .env	Connect Python to a public API	API integration

Weekly checkpoint: Explain the week's concepts without notes, demonstrate the deliverable, show Git commits, and list what is still unclear.

Week 2 — FastAPI, LLM & Prompt Design

Day	Learn	Hands-on	Deliverable
Day 8	FastAPI basics	Create /health, /users, /documents	FastAPI server
Day 9	Path/query params, Pydantic	Build validated APIs	Validation demo
Day 10	Async endpoints, DB integration	Connect FastAPI to DB	DB API
Day 11	LLM provider SDK/API	Build a basic chatbot	LLM chatbot
Day 12	System/user prompts, templates, few-shot	Create prompt templates	Prompt library
Day 13	Context window, temperature, hallucination	Run controlled prompt experiments	Experiment notes
Day 14	Streaming responses	Stream LLM output through FastAPI	Streaming chatbot

Weekly checkpoint: Explain the week's concepts without notes, demonstrate the deliverable, show Git commits, and list what is still unclear.

Week 3 — RAG Core

Day	Learn	Hands-on	Deliverable
Day 15	What RAG is and why it is needed	Draw and explain the RAG pipeline	Architecture diagram
Day 16	Document ingestion/loaders	Support PDF/TXT ingestion	Ingestion pipeline
Day 17	Chunking and overlap	Compare chunk sizes/overlap	Chunking experiment
Day 18	Embeddings	Generate embeddings for chunks	Embedding pipeline
Day 19	Chroma/FAISS vector DB	Store and query embeddings	Vector DB demo
Day 20	Cosine similarity, metadata filtering	Build semantic search	Search demo

Day	Learn	Hands-on	Deliverable
Day 21	Retrieve → prompt → generate	Connect retrieval to LLM	First RAG app

Weekly checkpoint: Explain the week's concepts without notes, demonstrate the deliverable, show Git commits, and list what is still unclear.

Week 4 · LangChain, Tools & Guardrails

Day	Learn	Hands-on	Deliverable
Day 22	LangChain basics, loaders, splitters	Rebuild RAG using LangChain	LangChain RAG
Day 23	Retrieval chains and prompt templates	Improve multi-document retrieval	Improved RAG
Day 24	RAG debugging and source display	Show retrieved chunks/sources	Traceable RAG
Day 25	Tool calling	Create calculator/database/search tools	Tool demo
Day 26	Guardrails and validation	Validate input/output; basic safety checks	Guarded RAG

Weekly checkpoint: Explain the week's concepts without notes, demonstrate the deliverable, show Git commits, and list what is still unclear.

Week 5 · LangGraph, Agentic RAG & Demo

Day	Learn	Hands-on	Deliverable
Day 27	LangGraph: state and graph flow	Build a simple graph workflow	LangGraph demo
Day 28	Conditional edges and loops	Create retrieval decision flow	Graph workflow
Day 29	Agentic RAG	Agent decides when/how to retrieve	Agentic RAG
Day 30	Demo project, testing and presentation	Demo, explain architecture, answer review questions	Demo project

Weekly checkpoint: Explain the week's concepts without notes, demonstrate the deliverable, show Git commits, and list what is still unclear.

Demo Project · Intelligent Document Assistant

Required flow: User → FastAPI → document ingestion → chunking → embeddings → vector DB → retrieval → LLM → answer + sources.

Minimum requirements

- Upload PDF/text documents
- Extract and chunk content
- Generate and store embeddings
- Perform semantic similarity search
- Use retrieved context in an LLM prompt
- Return an answer with source/chunk references
- Expose the workflow through a FastAPI endpoint
- Handle errors and keep API keys in .env
- Use Git with meaningful commits
- Prepare a short architecture explanation

Recommended additions: pytest-based tests, basic RAG evaluation, token/cost awareness, Docker basics, prompt-injection awareness and simple logging.